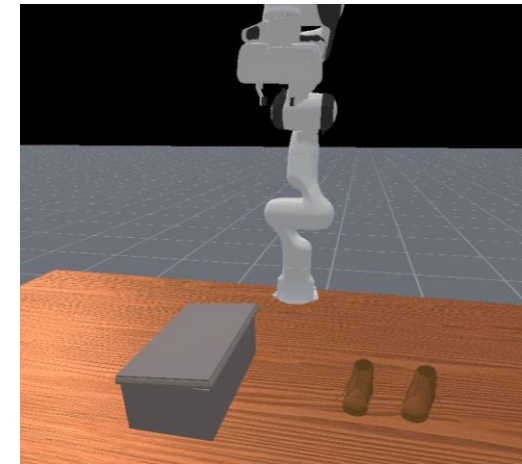
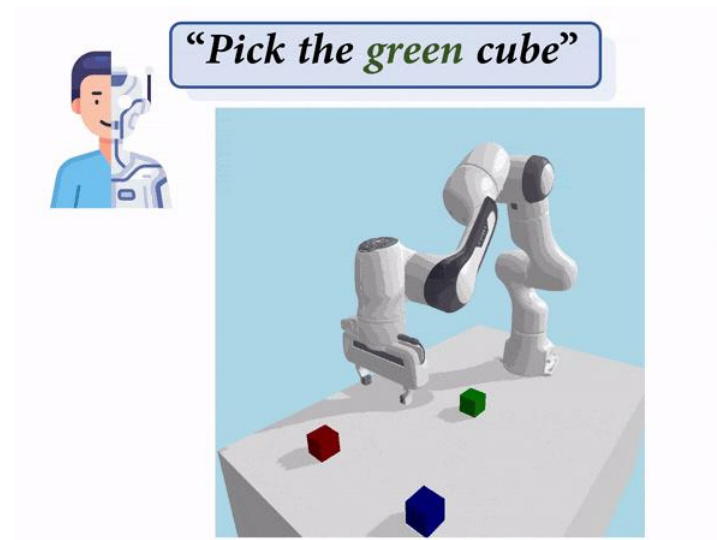
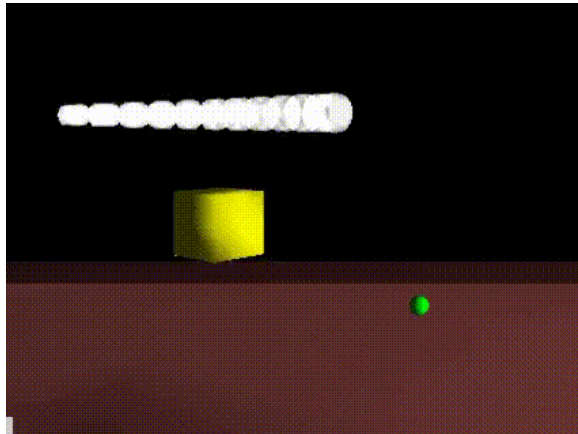
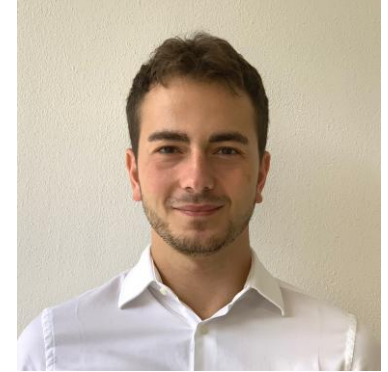


Robot Learning

Exercise 1 - Reinforcement Learning Fundamentals

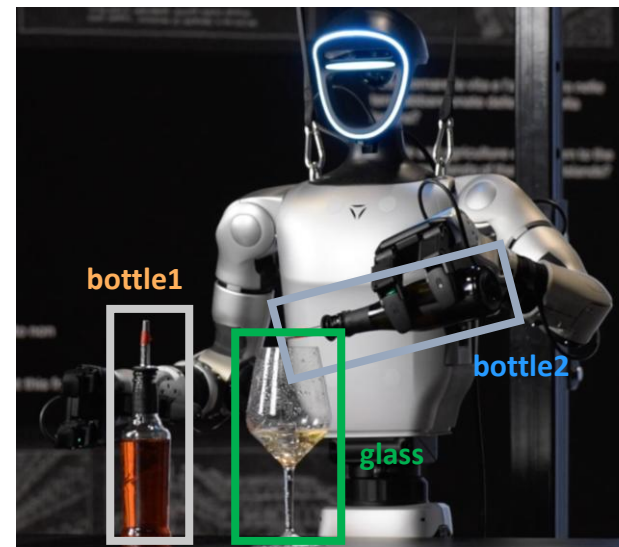
Andrea Protopapa

- PhD in Artificial Intelligence (3° year) - *Politecnico di Torino, Turin, Italy*
 - **Topics:** Robotics, Soft Robotics, Reinforcement Learning & Control, Imitation Learning, Inverse Reinforcement Learning, Human Robot Interaction



Davide Buoso

- PhD in Artificial Intelligence (2° year) - *Politecnico di Torino, Turin, Italy*
 - **Topics:** Robotics, Reinforcement Learning & Control, Planning and Reasoning for Manipulation, Generative Models

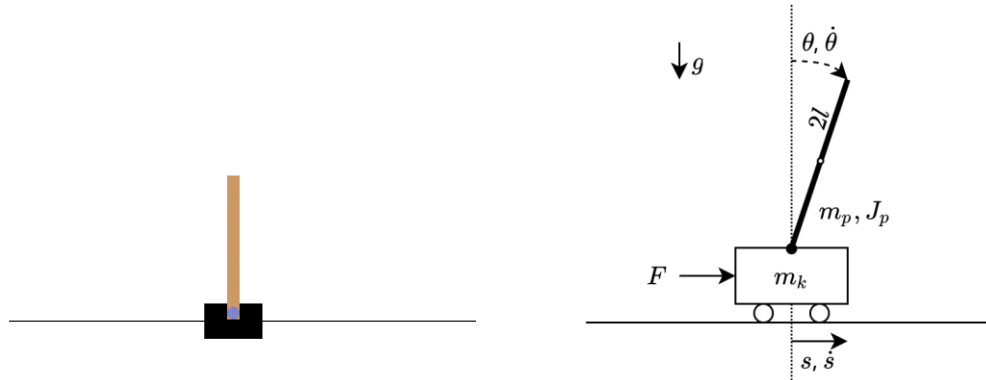


Topics of today lab

- CartPole environment, Gym
- Linear Quadratic Regulator (LQR)
- Fundamental concepts of Reinforcement Learning

CartPole environment, Gym

- **Gym**: a standard API for reinforcement learning, and a diverse collection of reference environments
- The Gym interface is simple, pythonic, and capable of representing general RL problems
- <https://gymnasium.farama.org/>



```
import gymnasium as gym

# Initialise the environment
env = gym.make("LunarLander-v3", render_mode="human")

# Reset the environment to generate the first observation
observation, info = env.reset(seed=42)
for _ in range(1000):
    # this is where you would insert your policy
    action = env.action_space.sample()

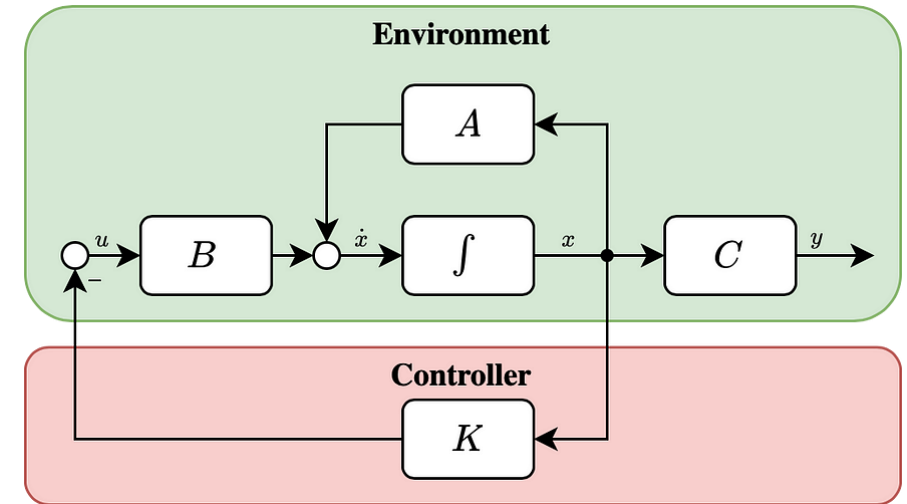
    # step (transition) through the environment with the action
    # receiving the next observation, reward and if the episode has terminated or truncated
    observation, reward, terminated, truncated, info = env.step(action)

    # If the episode has ended then we can reset to start a new episode
    if terminated or truncated:
        observation, info = env.reset()

env.close()
```

Linear Quadratic Regulator (LQR)

- Optimal control gain **K**
- We will *feedback* the states **x** of the environment and **K** will determine our input **u** into the system — the force *F*, that we want so apply onto the cart to balance the pole.
- Original system: nonlinear
 - Linearization
 - First order Taylor expansion around a desired working point



State equation

$$\dot{x} = Ax + Bu$$

$$\dot{x} = \begin{pmatrix} \dot{s} \\ \ddot{s} \\ \dot{\theta} \\ \ddot{\theta} \end{pmatrix} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{gm_p}{(m_k+m_p)\left(\frac{m_p}{m_k+m_p}-\frac{4}{3}\right)} & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & -\frac{g}{l\left(\frac{m_p}{m_k+m_p}-\frac{4}{3}\right)} & 0 \end{pmatrix} x + \begin{pmatrix} 0 \\ \frac{m_p}{(m_k+m_p)\left(\frac{m_p}{m_k+m_p}-\frac{4}{3}\right)} - 1 \\ -\frac{m_p}{m_k+m_p} \\ 0 \\ \frac{1}{l\left(\frac{m_p}{m_k+m_p}-\frac{4}{3}\right)} \end{pmatrix} u$$

Linear Quadratic Regulator (LQR)

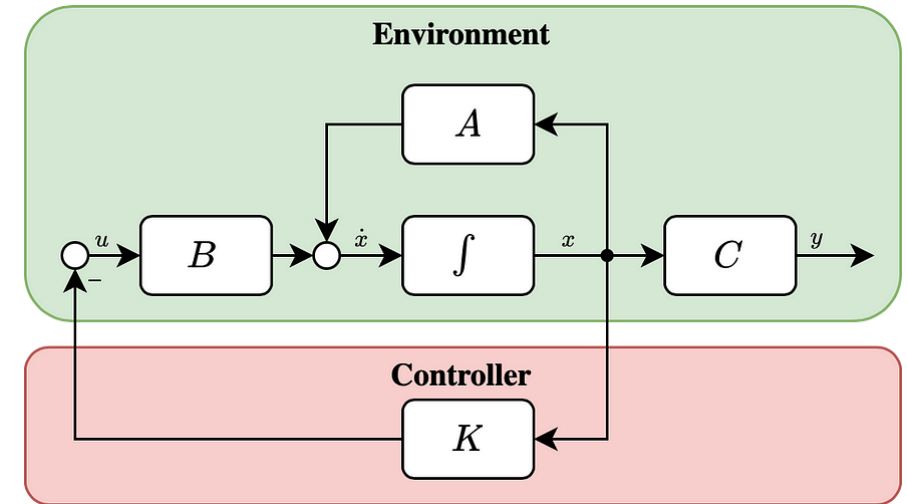
- LQR tries to find an optimal control matrix $\mathbf{K}$ by minimizing the quadratic cost function J

$$u = -Kx \quad s.t. \quad \min_{\mathbf{u}} J = \int_0^{\infty} \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u} \, dx$$

- $\mathbf{Q}$ (*state cost matrix*) states how important is the performance
- $\mathbf{R}$ (*control cost matrix*) states how expensive the usage of our control is
- Solution found by using the *Riccati differential equation*

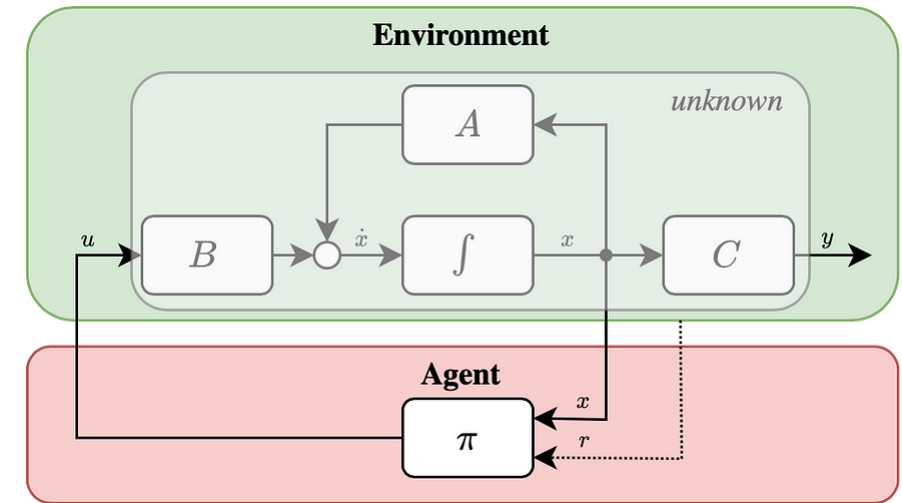
$$\mathbf{A}^T \mathbf{P} + \mathbf{P} \mathbf{A} - \mathbf{P} \mathbf{B} \mathbf{R}^{-1} \mathbf{B}^T \mathbf{P} + \mathbf{Q} = \mathbf{0}$$

$$\mathbf{K} = \text{inv}(\mathbf{R}) \mathbf{B}^T \mathbf{P}$$



Fundamental concepts of Reinforcement Learning

- In classic control we assume to **know** the underlying system dynamics, whereas in RL we **do not**
- The agent needs to learn to optimize the reward through **trial and error**



“What we want is a machine that can learn from experience.”
Alan Turing, 1947

Markov Decision Process

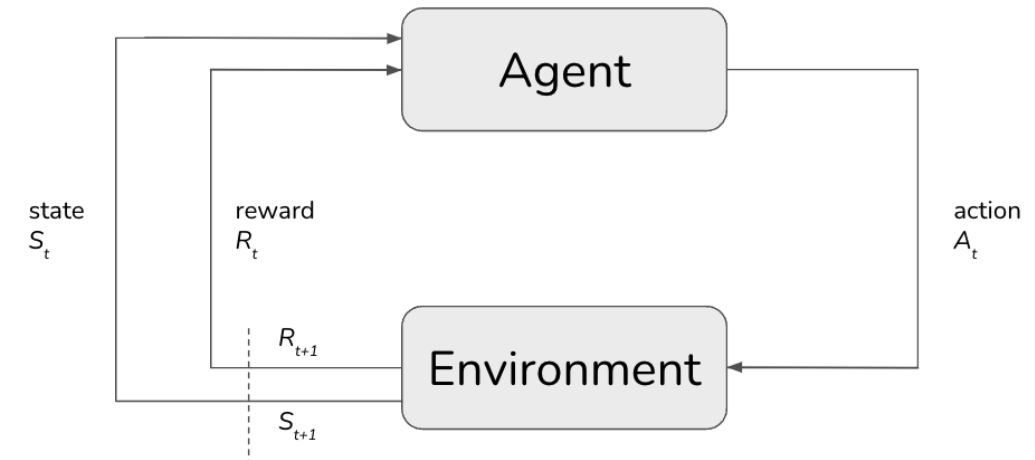
Given

- S : set of **states**
- A : set of **actions**
- R : reward probability distribution
 - A **reward** $R_t(s_t, a_t, s_{t+1})$ is a scalar feedback signal that indicates how well agent is doing at step t
- $P(s_{t+1} | s_t, a_t)$: transition probability distribution
- ρ_0 **initial state distribution** ($s_0 \sim \rho_0$)
- γ in $(0, 1]$: **discount factor**
- H : horizon over which the agent will act (number of steps in each episode)

Goal: find a **policy** $\pi^*(a_t | s_t)$ which **maximizes expected sum of rewards**

- The agent's behavior, a map from state s to action a

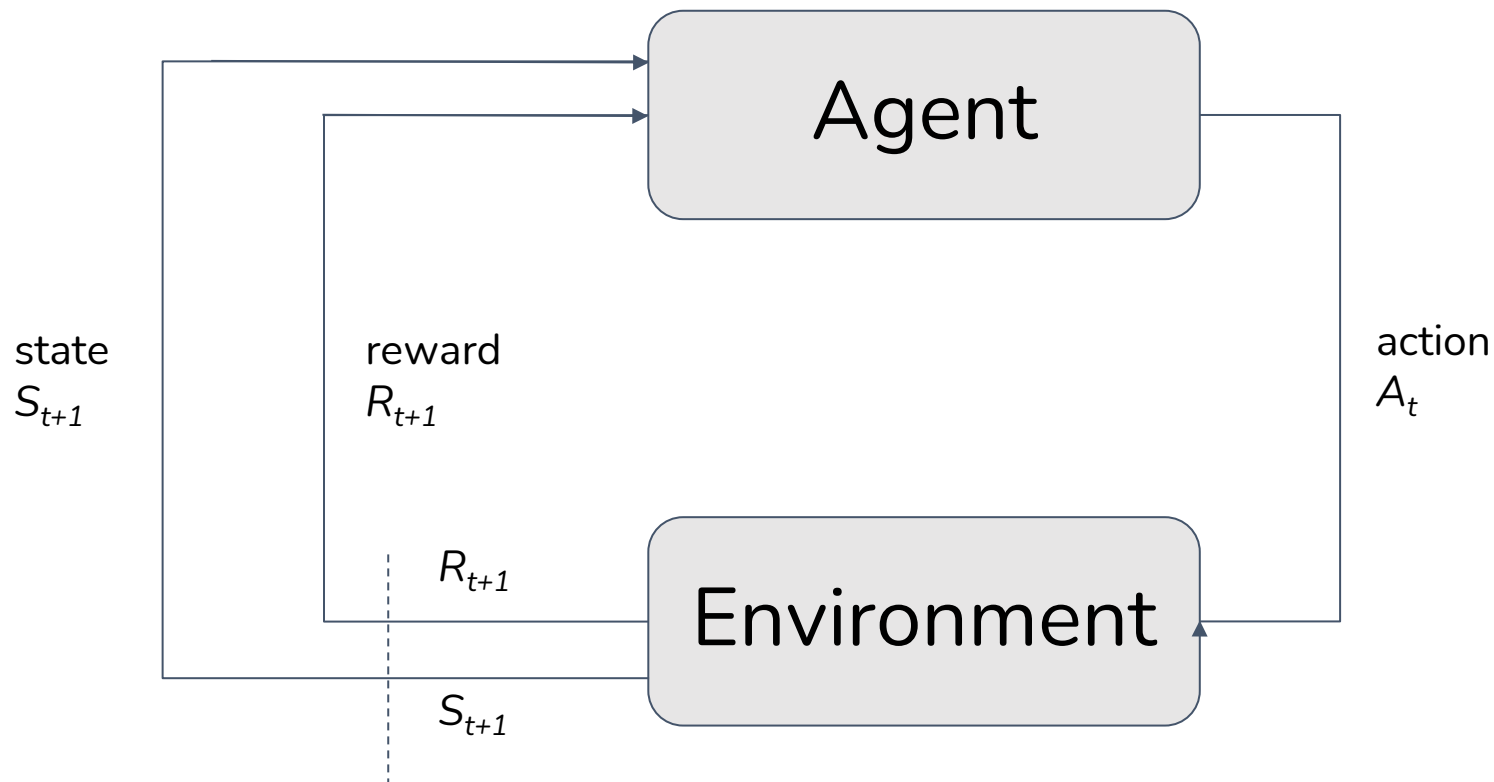
$MPD: \langle S, A, R, P, \rho_0 \rangle$



$$\pi^* = \operatorname{argmax}_{\pi} E \left[\sum_{t=0}^H \gamma^t R_t(S_t, A_t, S_{t+1}) | \pi \right]$$

Markov Decision Process

Trajectory: $s_0, A_0, R_1, s_1, A_1, R_2, \dots$



At each time step t :

- 1) **Agent** takes an action A_t
- 2) **Environment** updates given action A_t , emits state S_{t+1} and the reward R_{t+1}
- 3) The agent receives S_{t+1} and R_{t+1}

Policies

A **policy** is a rule used by an agent to decide what actions to take.

- **deterministic**: maps each state to a single action with certainty

$$a_t = \pi(s_t)$$

- **stochastic**: maps each state to a probability distribution over actions

$$a_t \sim \pi(\cdot | s_t)$$

- Different types: categorical, gaussian ...
- Two key computations
 1. sampling actions from the policy
 2. computing log likelihoods of particular actions, $\log \pi(a|s)$

Policies

A **policy** is a rule used by an agent to decide what actions to take.

- **deterministic**: maps each state to a single action with certainty

$$a_t = \pi_{\theta}(s_t)$$

- **stochastic**: maps each state to a probability distribution over actions

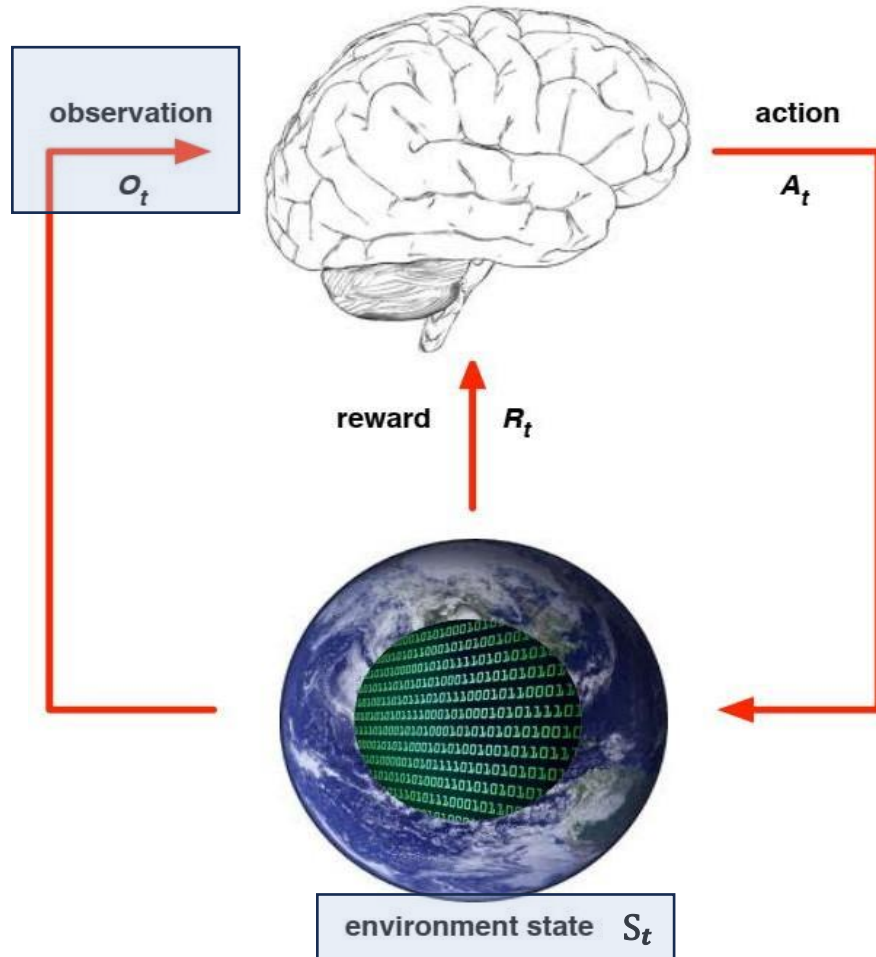
$$a_t \sim \pi_{\theta}(\cdot | s_t)$$

- Different types: categorical, gaussian ...
- Two key computations
 1. sampling actions from the policy
 2. computing log likelihoods of particular actions, $\log \pi_{\theta}(a|s)$

In deep RL, we deal
with **parameterized policies**

Hyperparameter tuning is
essential for good performance!

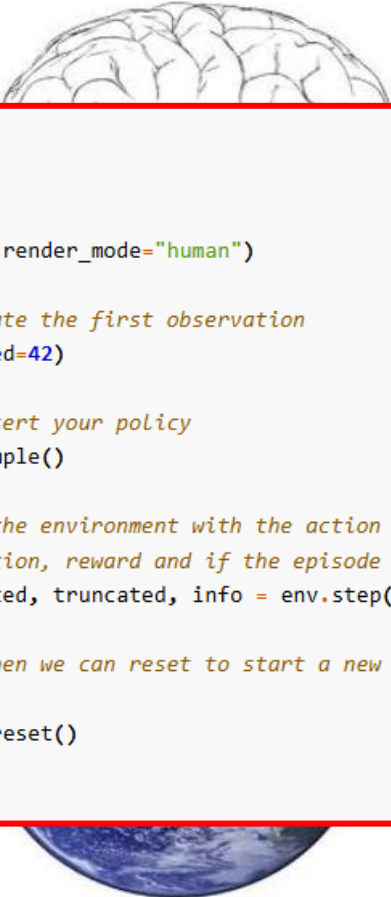
Agent and Environment



- State S_t : complete description of the state of the world.
- Observation O_t : a partial description of a state, which may omit information.

- ✓ At each step t the Agent:
 - ✓ Executes action A_t
 - ✓ Receives observation O_t
 - ✓ Receives scalar reward R_t
- ✓ The Environment:
 - ✓ Receives action A_t
 - ✓ Emits observation O_{t+1}
 - ✓ Emits scalar reward R_{t+1}
- ✓ t increments at environment step

Agent and Environment in Gym



```
import gymnasium as gym

# Initialise the environment
env = gym.make("LunarLander-v3", render_mode="human")

# Reset the environment to generate the first observation
observation, info = env.reset(seed=42)
for _ in range(1000):
    # this is where you would insert your policy
    action = env.action_space.sample()

    # step (transition) through the environment with the action
    # receiving the next observation, reward and if the episode has terminated or truncated
    observation, reward, terminated, truncated, info = env.step(action)

    # If the episode has ended then we can reset to start a new episode
    if terminated or truncated:
        observation, info = env.reset()

env.close()
```

environment state S_t

- ✓ At each step t the Agent:
 - ✓ Executes action A_t
 - ✓ Receives observation O_t
 - ✓ Receives scalar reward R_t
- ✓ The Environment:
 - ✓ Receives action A_t
 - ✓ Emits observation O_{t+1}
 - ✓ Emits scalar reward R_{t+1}
- ✓ t increments at environment step

Agent and Environment in Gym

```
import gymnasium as gym

# Initialise the environment
env = gym.make("LunarLander-v3", render_mode="human")

# Reset the environment to generate the first observation
observation, info = env.reset(seed=42)
for _ in range(1000):
    # this is where you would insert your policy
    action = env.action_space.sample()

    # step (transition) through the environment with the action
    # receiving the next observation, reward and if the episode has terminated or truncated
    observation, reward, terminated, truncated, info = env.step(action)

    # If the episode has ended then we can reset to start a new episode
    if terminated or truncated:
        observation, info = env.reset()

env.close()
```

<https://gymnasium.farama.org/>

Agent and Environment in Gym

```
import gymnasium as gym

# Initialise the environment
env = gym.make("LunarLander-v3", render_mode="human")

# Reset the environment to generate the first observation
observation, info = env.reset(seed=42)
for _ in range(1000):
    # this is where you would insert your policy
    action = env.action_space.sample()

    # step (transition) through the environment with the action
    # receiving the next observation, reward and if the episode has terminated or truncated
    observation, reward, terminated, truncated, info = env.step(action)

    # If the episode has ended then we can reset to start a new episode
    if terminated or truncated:
        observation, info = env.reset()

env.close()
```

<https://gymnasium.farama.org/>

Design a good reward function can be tricky

Agent and Environment in Gym

```
import gymnasium as gym

# Initialise the environment
env = gym.make("LunarLander-v3", render_mode="human")

# Reset the environment to generate the first observation
observation, info = env.reset(seed=42)

for _ in range(1000):
    # this is where you would insert your policy
    action = env.action_space.sample()

    # step (transition) through the environment with the action
    # receiving the next observation, reward and if the episode has terminated or truncated
    observation, reward, terminated, truncated, info = env.step(action)

    # If the episode has ended then we can reset to start a new episode
    if terminated or truncated:
        observation, info = env.reset()

env.close()
```

<https://gymnasium.farama.org/>

Agent and Environment in Gym

```
import gymnasium as gym

# Initialise the environment
env = gym.make("LunarLander-v3", render_mode="human")

# Reset the environment to generate the first observation
observation, info = env.reset(seed=42)
for _ in range(1000):
    # this is where you would insert your policy
    action = env.action_space.sample()

    # step (transition) through the environment with the action
    # receiving the next observation, reward and if the episode has terminated or truncated
    observation, reward, terminated, truncated, info = env.step(action)
    env.render()
    # If the episode has ended then we can reset to start a new episode
    if terminated or truncated:
        observation, info = env.reset()

env.close()
```

<https://gymnasium.farama.org/>

Agent and Environment in Gym

```
import gymnasium as gym

# Initialise the environment
env = gym.make("LunarLander-v3", render_mode="human")

# Reset the environment to generate the first observation
observation, info = env.reset(seed=42)
for _ in range(1000):
    # this is where you would insert your policy
    action = env.action_space.sample()

    # step (transition) through the environment with the action
    # receiving the next observation, reward and if the episode has terminated or truncated
    observation, reward, terminated, truncated, info = env.step(action)

    # If the episode has ended then we can reset to start a new episode
    if terminated or truncated:
        observation, info = env.reset()

env.close()
```

<https://gymnasium.farama.org/>

Agent and Environment in Gym

```
import gymnasium as gym

# Initialise the environment
env = gym.make("LunarLander-v3", render_mode="human")

# Reset the environment to generate the first observation
observation, info = env.reset(seed=42)
for _ in range(1000):
    # this is where you would insert your policy
    action = env.action_space.sample()

    # step (transition) through the environment with the action
    # receiving the next observation, reward and if the episode has terminated or truncated
    observation, reward, terminated, truncated, info = env.step(action)

    # If the episode has ended then we can reset to start a new episode
    if terminated or truncated:
        observation, info = env.reset()

env.close()
```

<https://gymnasium.farama.org/>

Attributes

Env.action_space: `spaces.Space[ActType]`

The Space object corresponding to valid actions, all valid actions should be contained with the space. For example, if the action space is of type `Discrete` and gives the value `Discrete(2)`, this means there are two valid discrete actions: 0 & 1.

```
>>> env.action_space
Discrete(2)
>>> env.observation_space
Box(-inf, inf, (4,), float32)
```

Env.observation_space: `spaces.Space[ObsType]`

The Space object corresponding to valid observations, all valid observations should be contained with the space. For example, if the observation space is of type `Box` and the shape of the object is `(4,)`, this denotes a valid observation will be an array of 4 numbers. We can check the box bounds as well with attributes.

```
>>> env.observation_space.high
array([4.8000002e+00, inf, 4.1887903e-01, inf], dtype=float32)
>>> env.observation_space.low
array([-4.8000002e+00, -inf, -4.1887903e-01, -inf], dtype=float32)
```

Gym Installation and other libraries

- Use [conda](#) environments

```
# pip install <lib>
gymnasium[classic-control, mujoco]==1.2.1
torch==2.8.0
seaborn==0.13.2
matplotlib==3.10.7
scipy==1.15.3
torch==2.8.0
```

```
# for cpu only: pip install torch==2.8.0 --extra-index-url https://download.pytorch.org/whl/cpu
```

Be also sure to have a compatible
version of python
(Python 3.10)